

Serializability

Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. **conflict serializability**
 2. **view serializability**

Schedules A and B in Figures 20.5(a) and (b) are called *serial* because the operations of each transaction are executed consecutively, without any interleaved operations from the other transaction. In a serial schedule, entire transactions are performed in serial order: T_1 and then T_2 in Figure 20.5(a), and T_2 and then T_1 in Figure 20.5(b). Schedules C and D in Figure 20.5(c) are called *nonserial* because each sequence interleaves operations from the two transactions.

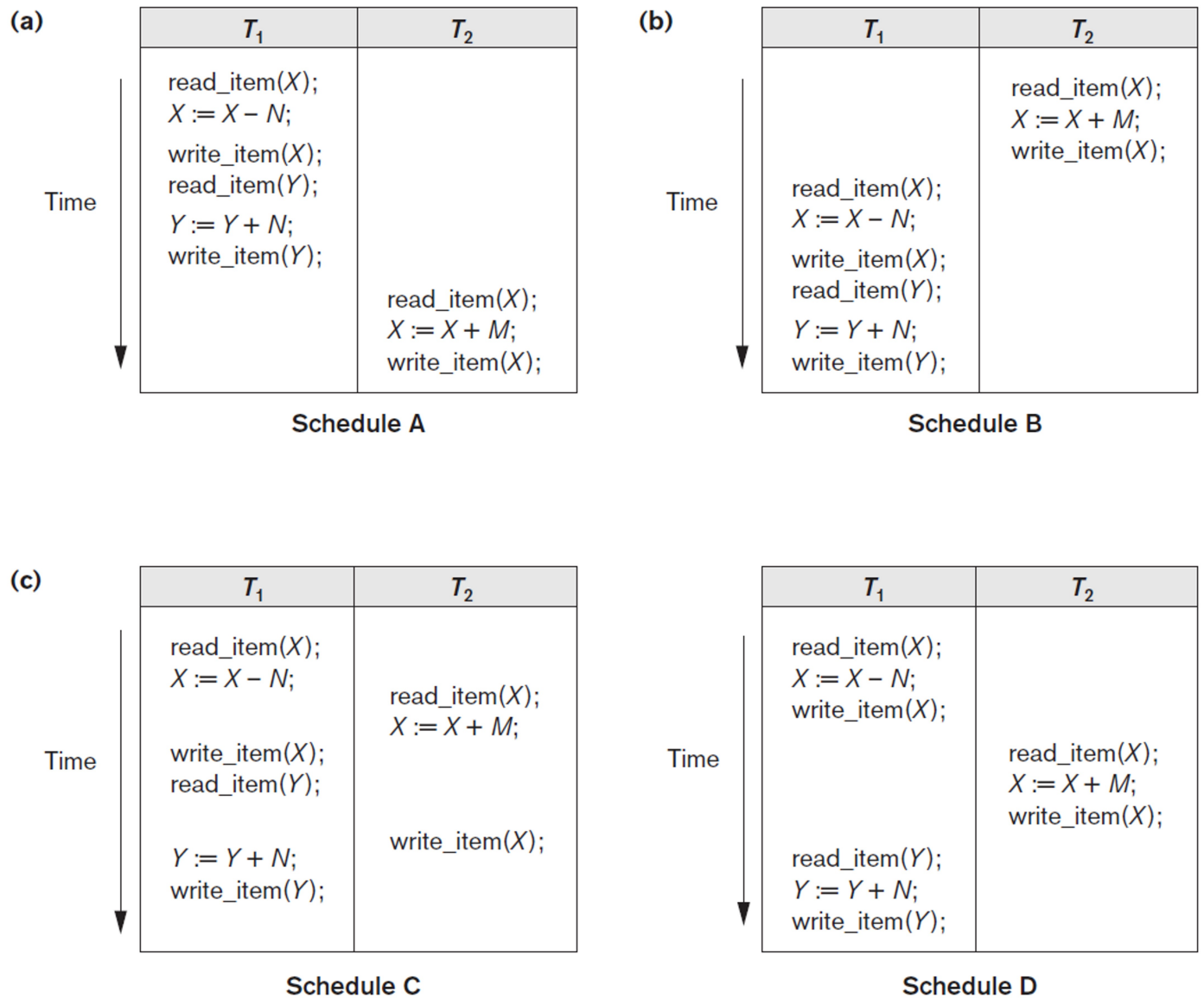


Figure 20.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c) Two nonserial schedules C and D with interleaving of operations.

Formally, a schedule S is **serial** if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule; otherwise, the schedule is called **nonserial**. Therefore, in a serial schedule, only one transaction at a time is active—the commit (or abort) of the active transaction initiates execution of the next transaction. No interleaving occurs in a serial schedule. One reasonable assumption we can make, if we consider the transactions to be *independent*, is that *every serial schedule is considered correct*. We can assume this because every transaction is assumed to be correct if executed on its own (according to the *consistency preservation* property of Section 20.3). Hence, it does not matter which transaction is executed first. As long as every transaction is executed from beginning to end in isolation from the operations of other transactions, we get a correct end result.

The problem with serial schedules is that they limit concurrency by prohibiting interleaving of operations. In a serial schedule, if a transaction waits for an I/O operation to complete, we cannot switch the CPU processor to another transaction, thus wasting valuable CPU processing time. Additionally, if some transaction T is long, the other transactions must wait for T to complete all its operations before starting. Hence, serial schedules are *unacceptable* in practice. However, if we can determine which other schedules are *equivalent* to a serial schedule, we can allow these schedules to occur.

To illustrate our discussion, consider the schedules in Figure 20.5, and assume that the initial values of database items are $X = 90$ and $Y = 90$ and that $N = 3$ and $M = 2$. After executing transactions T_1 and T_2 , we would expect the database values to be $X = 89$ and $Y = 93$, according to the meaning of the transactions. Sure enough, executing either of the serial schedules A or B gives the correct results. Now consider the nonserial schedules C and D. Schedule C (which is the same as Figure 20.3(a)) gives the results $X = 92$ and $Y = 93$, in which the X value is erroneous, whereas schedule D gives the correct results.

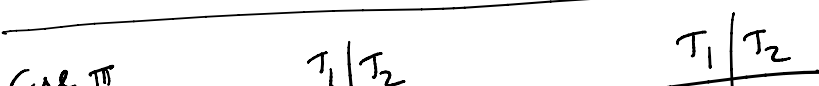
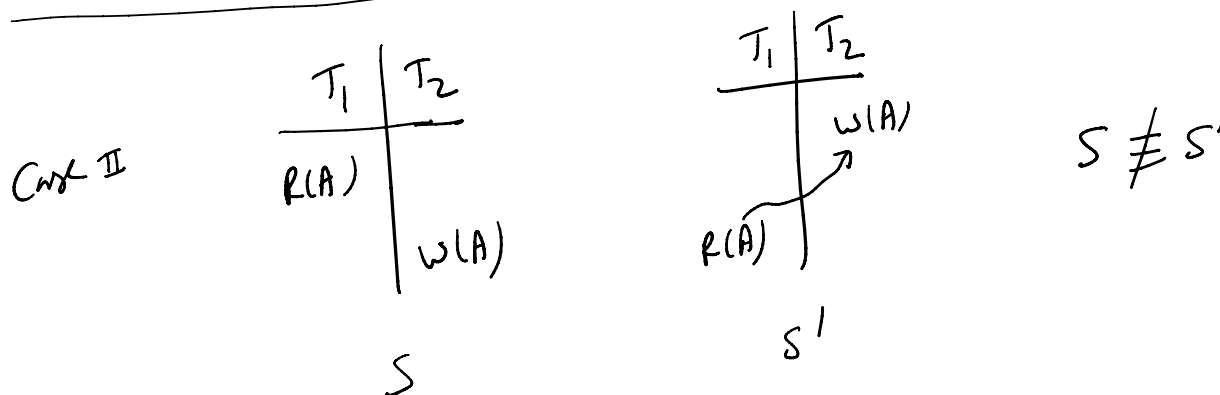
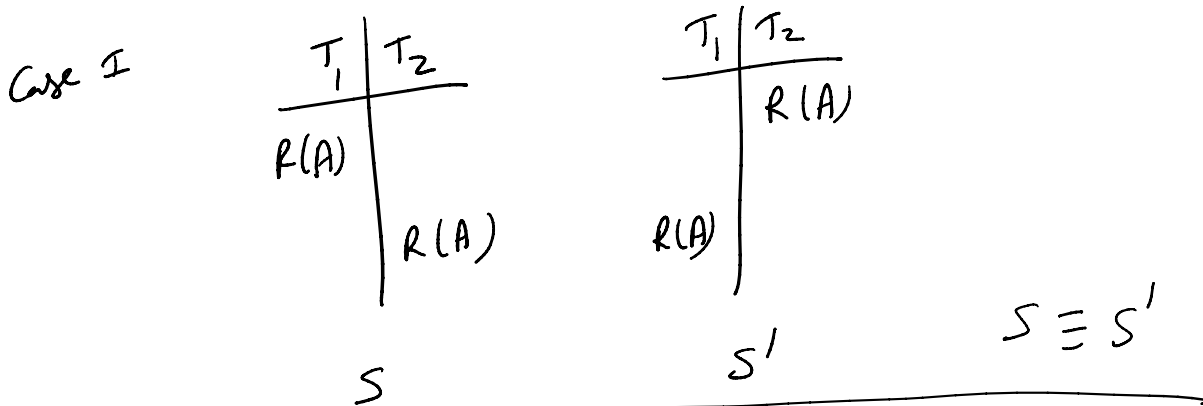
The definition of *serializable schedule* is as follows: A schedule S of n transactions is **serializable** if it is *equivalent to some serial schedule* of the same n transactions. We will define the concept of *equivalence of schedules* shortly. Notice that there are $n!$ possible serial schedules of n transactions and many more possible nonserial schedules. We can form two disjoint groups of the nonserial schedules—those that are equivalent to one (or more) of the serial schedules and hence are serializable, and those that are not equivalent to *any* serial schedule and hence are not serializable.

Conflict Serializability

Conflict Equivalence of Two Schedules. Two schedules are said to be **conflict equivalent** if the relative order of any two conflicting operations is the same in both schedules.

Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict.
 4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict.
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
 - If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.



Case III

T_1	T_2
$w(A)$	$R(A)$

S

T_1	T_2
$w(A)$	$R(A)$

S'

$S \neq S'$

Case IV

T_1	T_2
$w(A)$	$w(A)$

S

T_1	T_2
$w(A)$	$w(A)$

S'

$S \neq S'$

Case V

$T_1: R(A) | w(A)$

$T_2: R(B) | w(B)$

T_1	T_2
$w(A)$	$w(B)$

S

T_1	T_2
$w(A)$	$w(B)$

S'

$S \equiv S'$

$w(A)$ $w(B)$
 $R(A)$ $w(B)$
 $w(A)$ $R(B)$

Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are conflict equivalent.
- We say that a schedule S is conflict serializable if it is conflict equivalent to a serial schedule
- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2	T_1	T_2
read (A)		read (A)	
write (A)		write (A)	
	read (A)	read (B)	
	write (A)	write (B)	
			read (A)
read (B)			write (A)
write (B)			read (B)
	read (B)		write (B)
	write (B)		

Schedule 3 Schedule 6

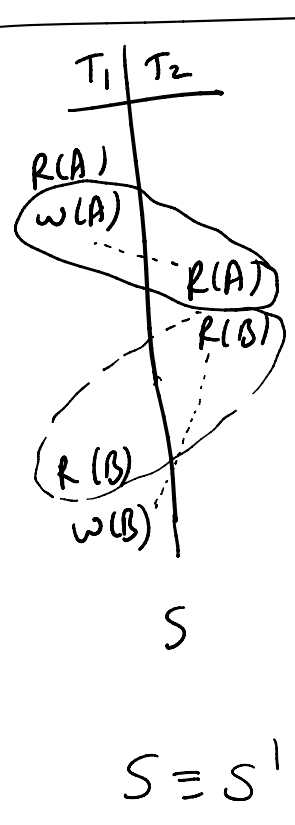
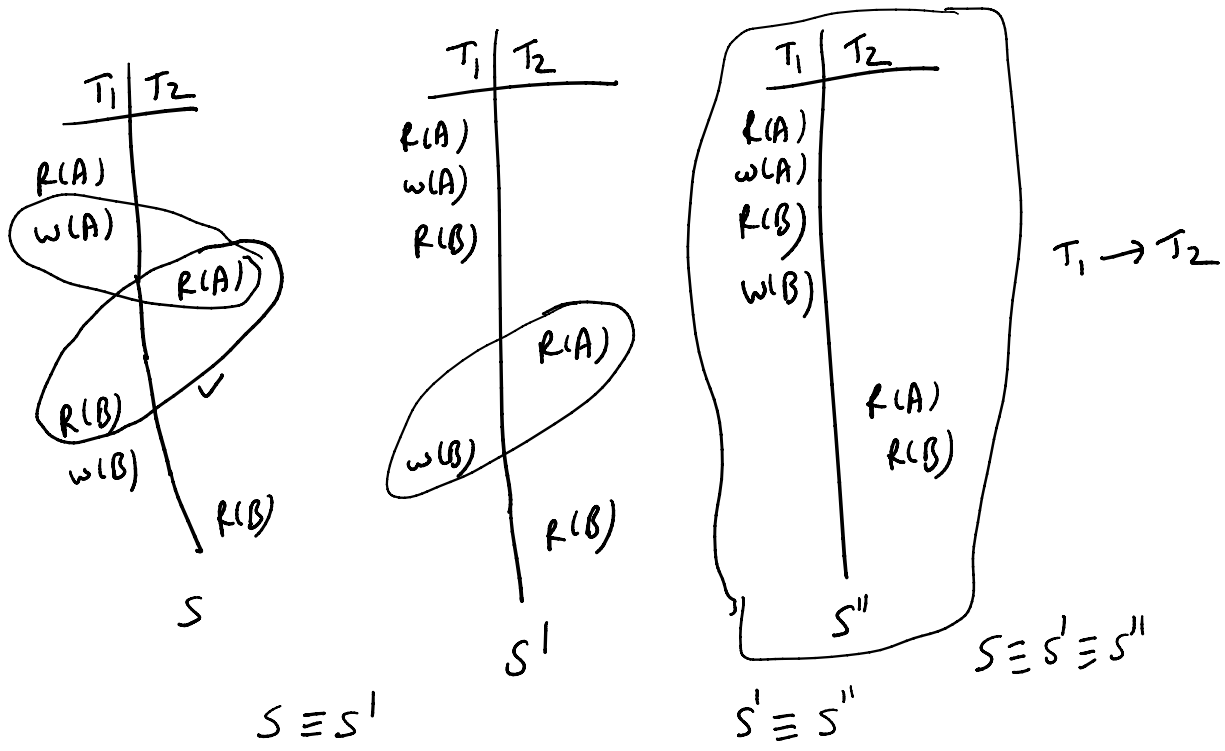
$T_1 \rightarrow T_2$ ✓

- Example of a schedule that is not conflict serializable:

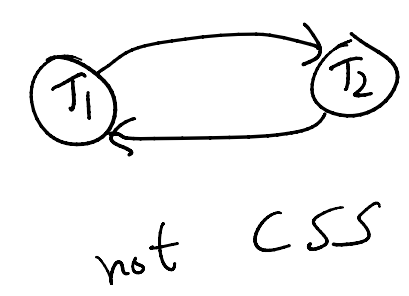
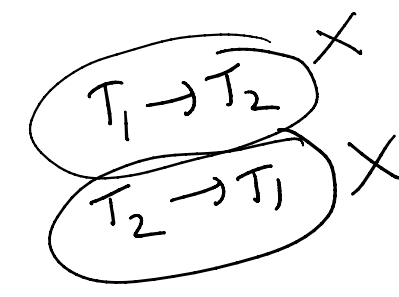
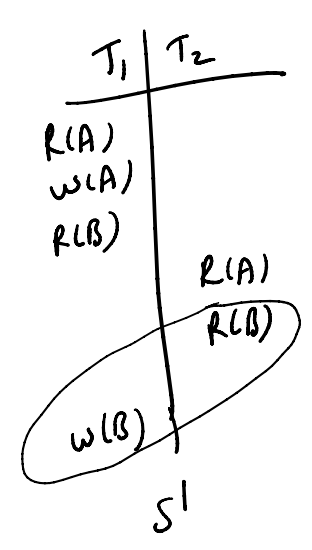
T_3	T_4
read (Q)	
	write (Q)
write (Q)	

$T_3 \rightarrow T_4$
 $T_4 \rightarrow T_3$ ✗

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



$T_1: A \xrightarrow{R_B^{500}} B$
 $T_2: \text{Sum}(A+B)$



Testing for Conflict Serializability of a Schedule

Algorithm 20.1 can be used to test a schedule for conflict serializability. The algorithm looks at only the `read_item` and `write_item` operations in a schedule to construct a **precedence graph** (or **serialization graph**), which is a **directed graph** $G = (N, E)$ that consists of a set of nodes $N = \{T_1, T_2, \dots, T_n\}$ and a set of directed edges $E = \{e_1, e_2, \dots, e_m\}$. There is one node in the graph for each transaction T_i in the schedule. Each edge e_i in the graph is of the form $(T_j \rightarrow T_k)$, $1 \leq j \leq n$, $1 \leq k \leq n$, where T_j is the **starting node** of e_i and T_k is the **ending node** of e_i . Such an edge from node T_j to node T_k is created by the algorithm if a pair of conflicting operations exist in T_j and T_k and the conflicting operation in T_j appears in the schedule *before* the *conflicting operation* in T_k .

Algorithm 20.1. Testing Conflict Serializability of a Schedule S

1. For each transaction T_i participating in schedule S , create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
3. For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
4. For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

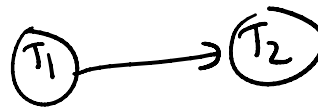
The precedence graph is constructed as described in Algorithm 20.1. If there is a cycle in the precedence graph, schedule S is not (conflict) serializable; if there is no cycle, S is serializable. A **cycle** in a directed graph is a **sequence of edges** $C = ((T_j \rightarrow T_k), (T_k \rightarrow T_p), \dots, (T_i \rightarrow T_j))$ with the property that the starting node of each edge—except the first edge—is the same as the ending node of the previous edge, and the starting node of the first edge is the same as the ending node of the last edge (the sequence starts and ends at the same node).

In the precedence graph, an edge from T_i to T_j means that transaction T_i must come before transaction T_j in any serial schedule that is equivalent to S , because two conflicting operations appear in the schedule in that order. If there is no cycle in the precedence graph, we can create an **equivalent serial schedule** S' that is equivalent to S , by ordering the transactions that participate in S as follows: Whenever an edge exists in the precedence graph from T_i to T_j , T_i must appear before T_j in the equivalent serial schedule S' .

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
 - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a topological sorting of the graph.

$$O(n^2)$$

$$O(n+e)$$



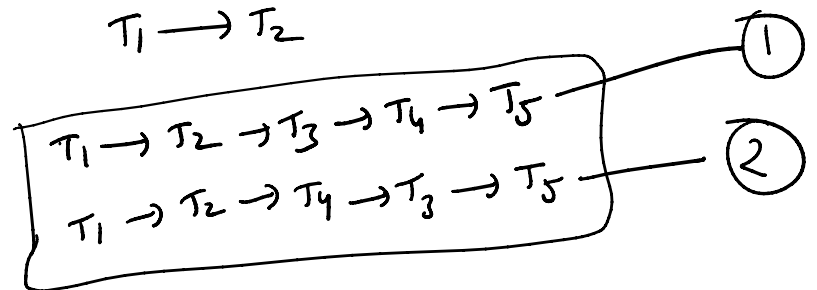
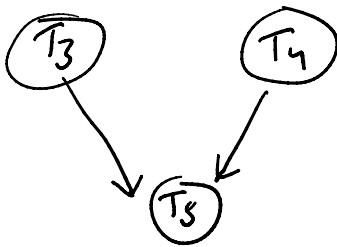
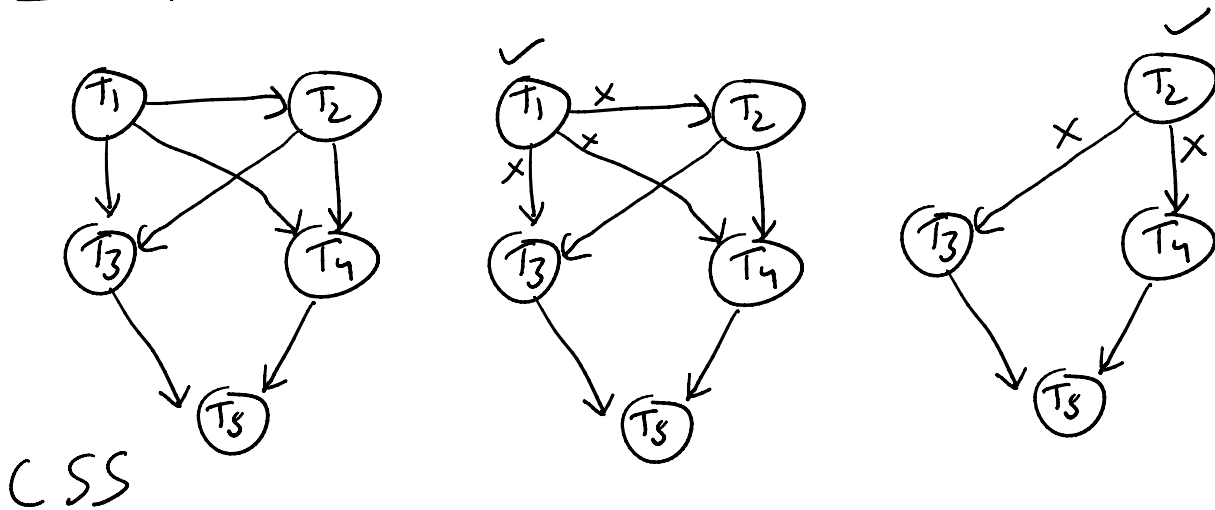
CSS

$$T_1 \rightarrow T_2$$

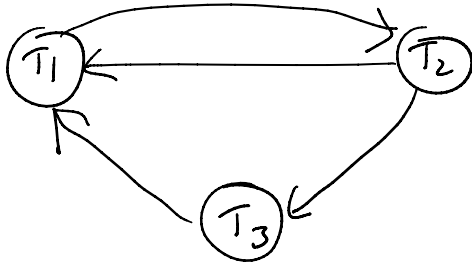
A schedule S is conflict serializable if and only if its precedence graph is acyclic. (An equivalent serial schedule in this case is given by any topological sort over the Precedence graph)

To find an equivalent serial schedule, start with a node that does not have any incoming edges, and then make sure that the node order for every edge is not violated.

Topological sort - for acyclic graph



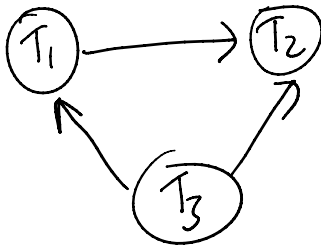
$$S_1: r_2(z) \ r_2(y) \ w_2(y) \ r_3(y) \ r_3(z) \ r_1(x) \ w_1(x) \ w_3(y) \ w_3(z) \\ r_2(x) \ r_1(y) \ w_1(y) \ w_2(x)$$



not
CSS

$$\begin{aligned} r_2(z) &= w_3(z) \\ r_2(y) &= w_3(y) \\ w_2(y) &= r_3(y) \\ w_2(y) &= w_3(y) \\ w_2(y) &= r_1(y) \\ r_3(y) &= w_1(y) \\ r_1(x) &= w_2(x) \\ w_1(x) &= r_2(x) \\ w_3(x) &= w_1(y) \\ &\vdots \end{aligned}$$

$$r_3(x) \ r_3(z) \ r_1(x) \ w_1(x) \ w_3(y) \ w_3(z) \ r_2(z) \ r_1(y) \ w_1(y) \ r_2(y) \\ w_2(y) \ r_2(x) \ w_2(x)$$



No cycle

CSS

$$T_3 \rightarrow T_1 \rightarrow T_2$$

$$\begin{aligned} r_3(x) &= w_1(x) \\ r_3(x) &= w_2(x) \\ r_1(x) &= w_2(x) \\ w_1(x) &= r_2(x) \\ w_3(y) &= r_1(y) \\ w_3(z) &= r_2(z) \\ &\vdots \end{aligned}$$